开源软件的价值

版权内容　略 不做翻译

作者：Manuel Hoffmann, Frank Nagle, Yanuo Zhou

摘要

评估一个非货币性（免费）产品的价值本身就是困难的。一个普遍的例子是开源软件（OSS），这是一种全球性的公共产品，在经济中扮演着至关重要的角色，并且是我们今天使用的大多数技术的基础。然而，由于 OSS 的非货币性质以及缺乏集中式的使用追踪，衡量其价值是很困难的。因此，OSS 在经济衡量中基本上未被计入。尽管先前的研究已经估算了重新创建这些软件的供给侧成本，但数据的缺乏阻碍了对 OSS 创造的、规模大得多的需求侧（使用）价值的估算。因此，为了解广泛使用的 OSS 所产生的完整经济和社会价值，我们利用了来自两个互补来源的独特的全球数据，这些数据捕捉了全球数百万家公司的 OSS 使用情况。我们首先通过计算重新创建一次最广泛使用的 OSS 的成本来估算供给侧价值。然后，我们基于每个使用该软件的公司（如果 OSS 不存在，则需要内部构建它）的重置价值来计算需求侧价值。我们估计，广泛使用的 OSS 的供给侧价值为 41.5 亿美元，但需求侧价值要大得多，为 8.8 万亿美元。我们发现，如果 OSS 不存在，公司在软件上的支出将需要比当前多 3.5 倍。我们样本中的前六种编程语言占 OSS 需求侧价值的 84%。此外，96%的需求侧价值仅由 5%的 OSS 开发者创造。

关键词：开源软件,全球公共产品

致谢：作者感谢 Linux 基金会提供的财政和行政支持，没有这些支持，来自“普查”的数据将无法获得。我们非常感谢哈佛商学院研究计算服务部、哈佛创新科学实验室、Linux 基金会以及软件组成分析数据提供商 Snyk、Synopsys 网络安全研究中心和 FOSSA 的支持。我们感谢 Tianli Li 和 Misha Bouziniar 出色的研究协助。我们还要感谢软件开发者 Boris Martinovic 以及微软的 Rich Lander 和 Scott Hanselman 对.NET 生态系统的见解。我们在哈佛商学院价值与估值会议、哈佛商学院 D 研究日以及 2023 年管理学会年会上收到了与会者的有益反馈。

1.引言

2011 年，风险投资家 Marc Andreessen 提出了著名的论点“软件正在吞噬世界”(Andreessen, 2011)，如今很少有人会反对这种观点。最近，风险投资家 Joseph Jacks 认为，“开源吞噬软件的速度比软件吞噬世界的速度更快”(Jacks, 2022)。其他近期研究也得出了类似结论，显示开源软件(OSS)——其源代码可公开供检查、使用和修改，通常以去中心化方式创建并免费分发——出现在 96%的代码库中(Synopsys, 2023)，并且一些商业软件高达 99.9%由免费可用的 OSS 组成(Musseau et al., 2022)。尽管在早期，OSS 经常复制现有专有软件的功能，但今天的 OSS 包括了各个领域的尖端技术，包括人工智能(AI)、量子计算、大数据和分析学。然而，尽管 OSS 对所有软件（进而对整个经济）的重要性日益增加，衡量其影响却一直难以实现。传统上，为了衡量一个商品或服务创造的价值，经济学家用价格(p)乘以销售数量(q)。但是，在 OSS 中，由于源代码是公开的，p 通常为零，并且由于代码复制和重用的限制很少，q 是未知的。例如，如果一家公司从公共代码仓库下载了一个 OSS 片段，

它可能在内部合法地复制数千次，然后与供应商或客户共享（也是合法的），因此公共下载数据是不够的。尽管最近的一些研究试图估算 p 的值（下文讨论），但用于估算 q 的数据对于仅仅少数 OSS 包来说是不可用或难以处理的。利用从多个来源新收集的数据，本文的目标是提供对 p 和 q 的估计，并利用这些来阐明一个问题：开源软件的价值是什么？

理解 OSS 的价值至关重要，不仅因为它在经济中扮演的角色，还因为它是数百年历史的经济学概念“公地”最成功和最具影响力的现代范例之一，而公地面临着被称为“公地悲剧”的命运风险。这个概念可以追溯到公元前 4 世纪的哲学家亚里士多德，他写道：“最多数人共有的东西得到的关心最少。人们最关注属于自己的东西，而对公共的东西则关心较少。”(Aristotle, 1981)。威廉·福斯特·劳埃德(William Forster Lloyd, 1833)通过强调用于放牧的共享土地的例子，在现代经济思想中重新提出了这个观点，每个牧民都有动机过度使用共享资源。加勒特·哈丁(Garrett Hardin, 1968)在一篇题为“公地悲剧”的文章中讨论这个问题时，将这个概念带入了更广泛的社会思潮。在此工作基础上，埃莉诺·奥斯特罗姆(Elinor Ostrom)因其研究突出了通过社区协调努力来避免公地悲剧的路径而获得诺贝尔奖，这些协调努力不需要政府强制实施的法律来管理和保护公地(Ostrom, 1990)。共享牧场和共享数字基础设施之间的相似之处是显而易见的——提供公共草地来喂养牛，进而养活人们，对农业经济至关重要；而不必重新编写别人已经写过的代码，这对现代经济至关重要。此外，在这两种情况下，尽管知道草地和代码是经济的关键投入，但衡量其实际价值却很困难。而且，正如著名数学家和物理学家开尔文勋爵(Lord Kelvin)据信所说的那样，“如果你无法衡量它，你就无法改进它。”如果你无法改进和维护它，这些公共物品可能会在其成功的重压下崩溃，因为它们被过度使用但投资不足(Lifshitz-Assaf and Nagle, 2021)。因此，衡量 OSS 创造的价值对于数字经济以及建立在它之上的其他经济的未来健康至关重要。

重要的是，最近的研究试图解决这些衡量问题，但在捕捉 OSS 使用的广度和深度方面都有所不足——我们试图用本文填补这一空白。例如，研究人员试图通过使用新颖的方法来估算美国现有 OSS 代码库的劳动力重置价值（2019 年为 380 亿美元）(Robbins et al., 2021)和欧盟创建的 OSS 代码库的劳动力重置价值（10 亿欧元）(Blind et al., 2021)来获得广度，这些方法是估算重写现有 OSS 所需的劳动力成本。这些努力在估算如果明天所有现有 OSS 都消失，重置它们需要多少成本方面做得非常好。然而，得出的估计依赖于两个重要的假设。第一，所有 OSS 从使用角度来看都具有同等价值；第二，OSS 的概念仍然存在，社会只需要重写一次代码，从而解决了上述 p 值缺失的问题，但没有解决 q 值缺失的问题。在一个 OSS 根本不存在的世界里，那么每个 OSS 软件不需要只重写一次，而是需要每个使用该软件的公司都重写一次（假设公司可以在其边界内自由共享软件）。其他研究(Greenstein and Nagle, 2014; Murciano-Goroff, Zhuo, and Greenstein, 2021)更深入地探讨了这个假设——尽管范围很窄——他们只关注了网络服务器（因为它们在互联网上是面向公众的，因此可以很容易地被衡量）。这两项研究使用不同的方法，都测量了这种单一类型软件的 q，并通过使用基于公司提供的闭源替代品价格的商品重置价值方法来估算 p。利用美国的数据，结果显示 2012 年 OSS Apache 网络服务器的价值为 20 亿美元(Greenstein and Nagle, 2014)，2018 年 Apache 和日益流行的 OSS 网络服务器 nginx 的总价值为 45 亿美元(Murciano-Goroff, et al.,

2021)。然而，尽管网络服务器是 OSS 生态系统的重要组成部分，但它们只占其中的一小部分。我们试图在现有研究的重要贡献基础上，力求既广泛又深入，以创建一个更完整的 OSS 价值衡量标准。

为了广泛而深入地考量 OSS 的价值，我们使用了两个主要数据源，它们使我们能够深入了解全球数万家公司使用的 OSS。第一个是“自由和开源软件普查 II-应用库”(Nagle et al., 2022)。普查 II 项目利用与多家软件组成分析(SCA)公司的合作关系，创建了多个最广泛使用的 OSS 列表。SCA 公司受雇扫描公司的代码库，以确保他们没有违反任何 OSS 许可证，并作为副产品，追踪其客户及其构建产品使用的所有 OSS 代码。普查 II 项目汇总了来自多个 SCA 的数据，基于数百万个数据点（OSS 使用观察值），构建了一个包含数万家公司 OSS 使用情况的数据集。第二个数据源是 BuiltWith 数据集，我们利用该数据集对近九百万个网站进行扫描，以识别这些网站部署的基础技术，包括 OSS 库。BuiltWith 数据已被用于多项学术研究(DeStefano & Timmis 2023, Dushnitsky & Stroube 2021, Koning et al., 2022)，但据我们所知，这是第一个关注 OSS 使用情况的研究。普查 II 数据和 BuiltWith 数据是互补的，因为前者关注内置于公司销售软件中的 OSS，而后者关注内置于公司网站中的 OSS，从而减少了数据集中观察值的重复计算风险。总的来说，这两个数据集的结合创建了迄今为止最完整的 OSS 使用量(q)衡量标准。此外，通过关注被公司广泛部署和使用的 OSS，而不是考虑 OSS 仓库中存在的所有项目，我们改进了先前研究的方法，减少了因那些作为公共 OSS 发布但实际上并未以任何实际方式使用的项目而产生的测量误差。不考虑测量误差导致的对 OSS 实际价值的高估，因为广泛使用的项目将与根本不使用的项目被以相同方式估价。

为了估算 p，我们遵循上述文献并使用劳动力重置价值。首先，我们通过测量给定 OSS 包中的代码行数，然后应用构造性成本模型 II(Boehm, 1984; Boehm et al., 2009)——也称为 COCOMO II——来估算从头编写代码所需的人时数，从而计算单个公司重新创建该 OSS 包的劳动力成本。然后，我们利用 Salary Expert 的全球工资数据，来准确估算如果这个 OSS 包不存在，公司将会承担的劳动力成本。这些成本可以与上述 OSS 包级别的 q 值相结合，以估算所有 OSS 在供给侧和需求侧的总价值。

我们发现，如果我们作为一个社会决定在供给侧重新创建所有广泛使用的 OSS，其价值范围在 12.2 亿美元到 62.2 亿美元之间。然而，考虑到 OSS 的实际使用情况，需求侧价值要大几个数量级，范围在 2.59 万亿美元到 13.18 万亿美元之间，前提是每个使用 OSS 包的公司都必须从头重新创建它（例如，OSS 的概念不存在）。我们记录了 OSS 价值在不同编程语言以及内部与外部面向编程工作之间的显著异质性。此外，我们发现程序员贡献的价值存在相当大的异质性，5%的程序员负责创造了供给侧和需求侧 90%以上的价值。我们使用的数据可以说是目前衡量公司使用 OSS 所创造价值的最全面的数据来源。然而，对于任何项目，证据都不是完整的，我们认为我们低估了价值，因为我们的数据，例如，不包括操作系统，这是一个重要的 OSS 遗漏类别。

本研究对学术文献、从业者和政策制定者做出了四个重要贡献。第一，它不仅考虑了 OSS 的供给侧（创建它的价格），而且以前所未有的规模考虑了使用/需求侧，从而提供了迄今为止对广泛使用的 OSS 价值的最完整估计。虽然先前对 OSS 价值的估计要么只求广泛（估算大量 OSS 的供给侧成本），要么只求深入（估算某一特定类型 OSS 创造的价值），但本研究通过使用独特的数据集，更好

地理解 OSS 使用的广度和深度，做到了两者兼顾。此外，本研究不是衡量所有 OSS 的价值，而是关注“公司使用 OSS 来创建其产品和网站”的价值，这限制了那些“无法确定哪些 OSS 实际用于生产”的研究中出现的测量误差。这一贡献建立并扩展了重要的研究（例如，Blind et al., 2021; Greenstein and Nagle, 2014; Murciano-Goroff et al., 2021; Robbins et al., 2021），这些研究试图确定这种对现代经济贡献巨大的重要资源的价值，尽管衡量这种贡献存在困难。在这样做时，它为一场与信息技术（IT）对生产力影响相关的长期讨论增加了见解(Brynjolfsson, 1993; Brynjolfsson and Hitt, 1996; Nagle, 2019a; Solow, 1987)，即“生产力悖论”，即 IT 投资对生产力统计数据的影响可能有限。这场辩论已经持续到新兴的 AI 领域(Brynjolfsson, Rock, and Syverson, 2018)。我们的工作通过强调 OSS 的存在所带来的巨大社会层面的成本节约（从而提升生产力），为这场对话做出了贡献。

其次，我们的研究通过强调与 OSS 投资相关的新颖数据来源，为无形资本的研究做出了方法论上的贡献。先前的研究表明，无形资本在经济增长(Corrado, Hulten, and Sichel, 2009)和公司价值(Peters and Taylor, 2017)中扮演着越来越重要的角色，但它常常未被衡量或被错误归因(Eisfeldt and Papanikolaou, 2014)。此外，我们展示了如何使用这些数据源来理解公司所做的真正软件投资，以及如果 OSS 不存在他们将不得不进行的投资。这很有价值，因为软件投资是一种日益重要的无形资本，正在推动创新(Branstetter, Drev, and Kwon, 2019)和绩效(Krishnan et al., 2000)。

第三，我们的结果有助于向公司和管理者强调 OSS 对其生产的重要性，并理想地增加支持 OSS 用户不应仅仅“搭便车”，还应参与 OSS 创建和维护的论点的分量（例如，Henkel, 2008; Nagle, 2018）。这些贡献只是如果 OSS 不存在公司将会承担的成本的一小部分，OSS 用户积极参与帮助维护他们使用的 OSS，对于 OSS 生态系统的健康和未来的福祉至关重要(Lifshitz-Assaf and Nagle, 2021; Zhang et al., 2019)。

第四，也是最后一点，我们的研究有助于为政策制定者提供信息，他们最近开始理解 OSS 对经济日益增长的重要性，并采取行动支持生态系统(European Commission, 2020; Executive Order No.14028, 2021)。然而，这些努力大多与保护现有 OSS 生态系统有关，这非常重要，但并没有进一步支持新 OSS 的创建。我们的结果有助于阐明 OSS 对整体经济的重要性，并增加呼吁社会更多支持这一关键资源的分量。我们的结果进一步表明，OSS 创造的大部分价值是由少数贡献者创造的。尽管人们早已知道少数 OSS 贡献者完成了大部分工作，但我们添加了新的见解，表明这对于广泛使用的 OSS 项目的价值创造更是如此，社会对这些个体的支持对于 OSS 乃至经济的未来成功至关重要。

本文的其余部分组织如下。第 2 节描述了实证背景和数据。第 3 节讨论了方法，包括测量挑战。在第 4 节中，我们估算了开源软件的价值。第 5 节是结论。

2.实证背景与数据

尽管自由和开源软件的概念自 1950 年代就已存在，但在 1980 年代由于 Richard Stallman 的努力以及他发起的 GNU 项目和自由软件基金会(Maracke, 2019)而变得更加流行。然而，直到 1990 年代，Linus Torvalds 发布了 Linux 内核（一个现在被广泛采用的 OSS 操作系统）之后，OSS 才真正起飞(Tozzi, 2016)。如今，OSS 被认为是数字经济的关键构建块，并被软件开发人员广泛用

于从手机到汽车再到冰箱以及尖端 AI 的各个领域(Lifshitz-Assaf and Nagle, 2021)。

我们使用两个互补的主要数据源来估算 OSS 的价值。第一个是“普查”(Census)，它是内向型的。它使我们能够识别进入公司创建产品的 OSS 代码。第二个数据集是 BuiltWith，它是外向型的，使我们能够识别消费者通过公司网站直接交互的 OSS 代码。由于两个数据集中的原始数据仅包含包名称、版本号和包管理器名称，不包含任何源代码相关信息，我们首先为每个包获取可公开访问的代码仓库，其中包括包级别的信息，如代码行数和使用的编程语言。人们可能会担心使用两个独立数据集进行价值计算会导致重复计算。然而，Census 和 BuiltWith 样本的重叠非常小：两个数据集中只发现了 18 个共同的包。此外，重复计算不太可能成为问题，因为这两个数据集捕捉了 OSS 使用的不同维度：Census 捕捉的是内向型的打包软件使用，而 BuiltWith 捕捉的是外向型网站中的使用。最后，作为一个补充性数据集，我们使用 GHTorrent，这是一个关于 GitHub（最流行的 OSS 托管平台，也是 OSS 研究常用的数据源）上 OSS 相关活动的详细历史记录（例如，Burton et al, 2017; Conti, Peukert, and Roche, 2023; Fackler, Hofmann, and Laurentsyeva, 2023; Kim, 2020; Tang, Wang, & Tong, 2023）。这个详细的历史数据使我们能够通过更好地理解价值在个体 OSS 开发者之间的分布，更深入地研究 OSS 的价值是如何创造的。我们在下面描述所有三个数据源的详细信息及其为估算所做的准备。

普查：自由和开源软件普查 II（下称：普查）由 Linux 基金会和哈佛创新科学实验室联合进行(Nagle et al., 2022)。普查是通过汇总来自三家主要软件组成分析(SCA)公司的数据创建的，这些公司在全球拥有数千家客户。SCA 公司受雇扫描客户的代码库，收集嵌入其专有软件中的 OSS 使用情况，以确保他们没有违反任何 OSS 许可协议。这通常作为与并购相关的尽职调查过程的一部分进行。与从公共来源（如包下载次数和代码更改）获得的其他 OSS 需求指标不同，这种数据收集方式确保我们观察到公司内部 OSS 使用的精确数量和类型。此外，它允许我们追踪每个包所依赖的依赖关系，以便我们能够观察到通常隐藏且难以获得的间接 OSS 使用情况。结果是，在 2020 日历年（1 月 1 至 12 月 31 日），SCA 客户公司在其产品中使用的 OSS 包有超过 270 万个观测值。

普查项目基于 libraries.io 的命名系统标准化了包名称，libraries.io 是一个由 Tidelift 维护的广泛使用的网站，整理了超过 800 万个开源包的信息。此次普查主要针对的是根据来自三个最知名的服务组件架构（SCA）供应商所报告的使用情况而得出的前 2000 个包，目的是为了确定使用最为广泛的开源软件，而非使用情况分布的长尾部分。这导致那些使用次数少于五次的软件包被剔除。由于用 JavaScript 编程语言编写的软件包（通常会托管在 Node 包管理器（NPM）上）通常比其他语言编写的软件包体积更小（代码行数更少），因此其使用次数往往更高（因为开发者通常会使用许多小型包，而非少数大型包），所以人口普查部门单独挑选出了 NPM 上排名前 1000 位的软件包以及非 NPM 托管的排名前 1000 位的软件包。

这个最终数据集覆盖了普查原始数据中观察到的总 OSS 使用量的 70%。对于普查中的这 2,000 个 OSS 包中的每一个，我们识别了托管在 GitHub（最广泛使用的 OSS 托管平台）上的原始代码。我们首先尝试从 libraries.io 获取每个包的 GitHub 仓库统一资源定位符(URL)。通过这种初始方法，我们成功将

1,657 个包匹配到了仓库。对于没有匹配仓库的 URL，我们遵循 Singh(2020)中的方法进行了谷歌搜索。更具体地说，对于每个未匹配的包，我们使用 Google API 搜索包名称和“GitHub 仓库”，并将结果中的第一个 GitHub 仓库 URL 视为其最佳匹配的 GitHub URL。这导致额外 174 个包匹配到了仓库。最后，我们手动搜索了剩余 169 个包的 URL，以识别相关的仓库。整个过程最终将 2,000 个 Census 包中的 1,840 个匹配到了包含该包原始源代码的代码仓库。未匹配的包被确定要么已从 GitHub 删除，要么已成为专有软件（因此原始源代码不再可用），因此，手动搜索使我们能够高度自信地从我们的分析中删除了 160 个未匹配的包（不到 Census 2,000 个样本的 8%）。

BuiltWith。BuiltWith 数据包含对全球所有公共网站的扫描，并识别它们使用的技术。与关注 OSS 使用的内向型 Census 数据不同，BuiltWith 数据扫描公司网站上专有软件和 OSS 的使用情况，但没有明确区分。为了将 BuiltWith 中的 OSS 与专有软件区分开，我们转向了技术类别中所有开源网络开发软件的子集，其中包括“JavaScript 及其库”，这生成了 778 个与 Census 数据中的 NPMOSS 类别相对应的观测值。选择这个样本有两个原因。首先，这个特定类别由 BuiltWith 构建，我们将其用作 OSS 的代理，以将其与货币化软件区分开。其次，JavaScript 是构建网站的核心技术之一，是 GitHub 上按使用量计算最流行的编程语言(GitHub, 2022)，从而使我们能够从需求侧角度捕捉最重要的 OSS。扫描包括 880 万个独特网站和 7280 万个相应的 OSS 使用观测值，时间跨度为 2020 年 1 月 1 日至 11 月 16 日。此外，为了确保我们衡量由私营部门产生的 OSS 使用价值，我们将 BuiltWith 中与 JavaScript 相关的 OSS 的采用域名与 Orbis、Compustat 和 PitchBook 中记录的公司网站进行匹配，这三个是常用的全球注册商业活动数据库。执行此匹配可确保非商业网站（例如，个人的个人网站）使用的 OSS 被排除在我们的分析之外。这导致了 38.6%的匹配率，对应于约 340 万个不同公司的网站。

对于 BuiltWith 数据，我们不能使用我们用于 Census 的第一种方法（使用 libraries.io 来帮助识别仓库 URL），因为 BuiltWith 只提供了与包相关的技术名称，而不包括其他信息（例如，包和包管理器名称）。因此，我们从执行上述谷歌搜索方法开始，该方法将 695 个包匹配到仓库。然后，我们手动搜索了剩余 83 个未匹配包中的 46 个的 GitHub URL，结果增加了 46 个包。总的来说，对于 BuiltWith 数据，我们能够在 778 个软件包库匹配项中识别出 741 个。与 Census 数据一样，剩余的 37 个未匹配包（不到 5%）因为已从 GitHub 删除而被从我们的分析中删除。

GHTorrent 数据库。为了衡量 OSS 价值创造的分布情况，我们利用了 GHTorrent 数据库，该数据库包含了使用 GitHub 的表述性状态传递(REST)应用程序编程接口(API)获取的 GitHub 上的全部活动历史记录。我们利用其关于 GitHub 仓库、开发者级别的提交以及开发者公开资料信息的记录来估算每个开发者的贡献。我们分两步缩小了开发者贡献分析的样本范围。首先，我们根据我们的 Census-BuiltWith 联合样本的仓库 URL，从 GHTorrent 中筛选了 GitHub 仓库及其提交记录。其次，我们通过移除大约 8000 个（12%）我们认为是机器人的 GitHub 贡献者，专注于人类对 OSS 的贡献。最终样本包含约六万名开发者和 230 万次提交。

为了将这些三个数据集准备好进行估算，我们首先使用 OSS 包 pygount（用于计算代码行数）和 linguist（用于识别编程语言）确定了每个包的代码

行数和使用的编程语言。我们将每种不同的语言分类为三个不同的桶（bucket）之一，从更可能是人类编写的到更可能是机器编写的（见表 A1）。桶 1 包含编程和标记语言（最有可能是人类编写的），桶 3 包含数据（最有可能是机器编写的），桶 2 包含介于两者之间的任何内容，例如配置文件（有时是人类编写的，有时是机器编写的，但仅通过查看代码很难判断）。在我们的主要估算中，我们只使用桶 1，同时在附录中提供桶 2 和桶 3 的稳健性检验，因此我们的结果代表了一个下限。最后，对于一些分析，我们更深入地展示了排名前 5 的编程语言（由 GitHub, 2022 针对 2020 年分类；我们的数据来自那一年）。排名前 5 的编程语言包括 C（包括 C#和 C++）、Java、JavaScript、Python 和 TypeScript。我们还将 Go 添加到此顶级语言列表中，因为它是一种在 OSS 中使用日益广泛的语言。

3.方法论

我们首先通过考虑 OSS 的供给侧和需求侧，使用劳动力市场方法来衡量 OSS 的价值。思想实验是，我们生活在一个 OSS 不存在的世界里，每个使用特定 OSS 包的公司都必须重新创建它。使用劳动力市场方法，我们计算每个 OSS 包的劳动力重置成本。为了估算每个包的价值，我们在单个包级别使用 COCOMO II(Boehm, 1984; Boehm et al., 2009)，然后对所有包的价值求和，以获得供给侧劳动力市场重置价值。然后，我们通过公司使用每个包的次数来缩放供给侧价值，同时移除每个公司内的多次使用，以获得需求侧价值。

其次，我们超越总量，考察价值创造过程中的不平等性。在 OSS 中，与许多创造性活动一样，通常是一小部分个人提供了大部分贡献，而许多其他人只做了一点点贡献（有时被称为 80/20 法则，意味着 80%的工作由 20%的人完成）。研究表明，这些频繁的贡献者常常因其努力而在 OSS 社区中获得有影响力的地位(Hanisch, et al, 2018)。因此，为了更好地理解价值创造在开发者之间的分布情况，我们首先使用 GHTorrent 数据，通过两种方式识别个体开发者的贡献：a)通过他们的 OSS 价值直接贡献，b)通过他们贡献的仓库总数。然后，我们测试这两种贡献指标的集中程度，以更好地了解是许多开发者还是少数开发者贡献了我们衡量的总价值。我们在下面详细解释劳动力市场方法和价值评估的不平等性。

3.1 劳动力市场方法

对于劳动力市场方法，我们通过计算一个包的重置价值来估算 OSS 的价值。我们询问通过雇佣程序员并支付他们有竞争力的市场工资来复制该包需要多少成本。为了估算这个供给侧价值（V_S^{Labor}），我们采用上述完整的 OSS 包列表，然后计算每个唯一包中的代码行数。对于每个独特的包，i，我们计算其价值，然后对这些值求和以获得总价值：

$$V_S^{Labor} = \Sigma^N_{i=1} * V_{Si} = \Sigma^N_{i=1} P_i * 1 \qquad (1)$$

在这个计算中，我们没有纳入任何生产外部性，因为我们假设从一个包到另一个包没有会降低编程成本的知识溢出。这种方法与其他估算 OSS 供给侧成本的论文中使用的方法相似(Blind et al., 2021; Nagle, 2019b; Robbins et al., 2021)。然后，我们通过纳入每个包的使用信息(Q)来计算 OSS 的需求侧价值：

$$V_D^{Labor} = \Sigma^N_{i=1} * V_{Di} = \Sigma^N_{i=1} P_i * Q_i \qquad (2)$$

在这里，我们没有纳入消费外部性，即，我们不允许当一个包被创建时公众产生普遍利益，并且我们进一步确保每个公司只替换他们使用的一个包一次，因为替换后的包可以在公司内部作为俱乐部商品使用（例如，参见 Cornes and Sandler, 1996）。价值（V_S^{Labor}）反映了重写所有广泛使用的 OSS 一次的成本（例如，OSS 的概念仍然存在，但所有这些包都需要从头重写），而价值（V_D^{Labor}）反映了每个使用这些 OSS 包之一的公司支付给开发者重写这些包的成本（例如，OSS 本身不再存在）。

对于供给侧和需求侧模型，我们通过构造性成本模型 II（也缩写为 COCOMO II）获得每个包(P_i)的美元价值(Boehm 1984, Boehm et al. 2009)。该模型先前已被美国国防部用于估算软件项目成本，以及用于先前估算 OSS 价值的研究中(Blind et al., 2021; Nagle, 2019b; Robbins et al., 2021)。它是一个高度灵活的模型，允许我们创建代码行数到美元价值的非线性转换。它使用以下建模方程：

$$E_i = \alpha \eta L_i^{\beta} \qquad (3)$$

在这其中，L 表示每项项目的代码行数（以千为单位），E 表示每个项目所需的人月工作量单位。与 Blind 等人（2021 年）的研究一致，我们使用默认的参数值 α、β 和 η。参数 α、β 是非线性调整因子，分别设置为 2.94 和 1.0997。参数 η 是一个工作量调整因子，可以根据对产品、硬件、人员和项目属性的主观评估进行修改。由于我们没有每个项目的先验数据，所以我们将 η 设置为默认值 1。为了得到每个开源软件项目的价格（P_i），我们随后将方程（3）的结果乘以一个平均程序员在开放市场上可以获得的加权全球工资。为了计算全球工资，我们包括了 2021 年根据 GitHub 开发者数量排名前 30 的国家的软件开发人员的基本月薪（Wachs 等人，2022 年）（每个国家的权重是其活跃的 GitHub 贡献者在前 30 个国家总贡献者中的占比）。我们还通过利用低工资水平（印度）和高工资水平（美国）的劳动力市场来设定界限，以更好地了解工资水平的变化情况，从而更清楚地了解使用哪些程序员群体来重新创建整个开源软件项目时，其价值会如何变化。

3.2 贡献衡量

为了更清晰地了解价值是如何产生的，以及这种价值的产生是否是均等的还是不均等的，我们分三个步骤构建了一个图形描述。在第一步中，我们衡量开发者的价值贡献。在第二步中，我们获取开发者所贡献的存储库数量的衡量标准。最后，我们使用大家熟知的洛伦兹曲线这一概念，为这两者提供一个图形表示，以便更好地理解贡献中的不平等程度。下面我们将详细介绍相关细节。

价值贡献。我们计算了每个开发者贡献的 OSS 供给和需求价值。在仓库层面，我们通过计算开发者的提交次数占仓库总提交次数的比例来量化每个开发者的工作贡献。随后，将此比例分别乘以仓库的需求和供给价值，以得出该个体贡献者对该仓库的增加值贡献。最后，我们汇总每个开发者在所有仓库中的价值贡献。来自唯一开发者 Dev，V^{DEV}_J的个体价值贡献可以表示为：

$$V_j^{Dev} = \sum_i^N \sigma_i^{Dev} * V_{ij} \qquad (7)$$

其中 σ_i^{Dev} 是焦点开发者在代码仓库 i 中的提交次数占比，V_{ij} 是公式(1)和

(2)中定义的整个代码仓库 ii 的需求或供应价值，其中 j∈{D,S}，而 N 是我们主要样本（即 Census 和 BuiltWith 合并后的数据）中的代码仓库数量。

代码仓库贡献。这指的是给定开发者贡献的代码仓库数量，表示如下：

$$N^{Dev} = \sum_i^N \mathbb{1}\{\sigma_i^{Dev} > 0\} \tag{8}$$

其中$\mathbb{1}$是指示函数，当开发者对代码仓库 i 的提交次数非零时等于 1。该指标衡量了单个开发者所满足的多样化 OSS 需求。结合价值贡献指标，它有助于我们了解整体产生的价值是否集中在少数开发者手中。如果每个开发者参与多个代码仓库而不仅仅是少数几个，对整个 OSS 生态系统及其多样性来说可能更可取。

衡量贡献的分散程度。为了以图形方式检验开发者贡献值的分散程度，我们针对需求侧和供应侧价值都使用了洛伦兹曲线（Lorenz, 1905）。洛伦兹曲线是表示不平等程度的一种成熟方法，因此，它能让我们更好地理解开发者对私营经济中 OSS 的贡献分散程度。我们根据开发者对 OSS 需求和供应的贡献（如公式(7)所述）将其按升序系统排列。随后，将这些排名归一化到 0 到 100 百分位的范围内，作为洛伦兹曲线的 x 轴值。y 轴则显示相应的价值贡献 V_j^{Dev}。结果部分中的图形表示将阐明开发者价值贡献的不平等程度。为了补充分析，我们进一步通过绘制公式(8)来研究代码仓库贡献 N^{Dev} 的分散程度。这使我们能够确定任何显著的价值不平等是否源于顶级贡献者主要专注于少数异常受欢迎的代码仓库，或者源于他们参与了更广泛成功的代码仓库。

4.结果

应用 COCOMO II 的劳动力市场方法后，我们得到了 OSS 价值的全球估计值。为了计算总价值，我们首先需要底层的代码行数（用于计算供应侧价值），然后是使用统计数据（用于需求侧价值）。由于不同编程语言的价值可能存在显著的异质性，我们还展示了我们研究期间（2020 年）的主要编程语言，如上所述。

此处插入表一

表 1 显示了来自两个数据集的描述性统计数据。内向型的 Census（面板 A）包含略超过 2.617 亿行代码，其中 72%的行数归因于主要编程语言。平均每个包都包含 14.2 万行代码，而主要语言子集的平均行数略低，为 11.3 万行。在考虑需求（使用）侧时，我们观察到 Census 中的包被使用了超过 270 万次，其中 92%的使用量归因于主要语言。平均每个包被使用了 1,472.4 次，而主要语言的使用次数更高，约为 1,497.5 次。对于外向型的 BuiltWith 数据，我们发现了类似的模式，但数值水平不同。BuiltWith 中包含的包有超过 8200 万行代码，其中 71%归因于主要编程语言。BuiltWith 样本中的平均每个包有 11.1 万行代码，而主要语言的平均行数较低，约为 8 万行。这是因为我们的 BuiltWith 数据主要由基于 JavaScript 的包组成，这些包通常比用其他语言编写的包要小。BuiltWith 中的包被使用了超过 14.2 万次，其中 99.97%归因于主要语言。接下来，我们使用这些原始观测值，通过劳动力市场方法计算所有 OSS 的价值，并估算了从供应侧和需求侧创造的价值。

此处插入表二

表 2 显示了基于与公司相关的 Census 和 BuiltWith 联合样本的 OSS 价值估计值。表 2 中的所有估计值仅基于表 A1 中分类为桶 1 的软件语言，这些语言最

可能是由人类而非机器编写的。[19]第一列分别包含了来自低收入国家（印度）、全球平均工资和高收入国家（美国）的工资估计值（如上所述）。要使用来自印度的平均工资水平的程序员重新编写一次所有广泛使用的 OSS（例如，OSS 的概念仍然存在，但所有现有的 OSS 都被删除，需要从头编写），需要投资 12.2 亿美元。相比之下，如果使用美国的平均工资水平的程序员，重新编写所有广泛使用的 OSS 将需要投资 62.2 亿美元。使用来自世界各地的程序员池，并根据上文讨论的现有对 OSS 的地理贡献进行加权，其投资额将介于低收入和高收入国家之间，即 41.5 亿美元。将这些数字与类似研究中的数字进行比较，有助于理解所有 OSS 的价值（先前研究）与我们研究中广泛使用的 OSS 的价值之间的差异。Robbins 等人（2021）和 Blind 等人（2021）使用了与我们类似的方法，估计 2019 年美国创造的 OSS 价值为 380 亿美元，2018 年欧盟创造的 OSS 价值为 10 亿欧元。Wachs 等人（2022）显示，大约 50%的 OSS 贡献来自美国和欧盟的总和。总的来说，这些研究给出的 OSS 全球价值约为 780 亿美元。因此，我们仅针对面向公司且广泛使用的 OSS 的供应侧价值中间估计值（41.5 亿美元）是一个可信的下限，并突显了在不考虑给定 OSS 包是否被广泛使用的情况下，所有 OSS 的供应侧总价值估计值更高。这进一步表明，最广泛使用的 OSS 的供应侧价值大约是所有 OSS 供应侧价值的 5.5%。

表 2 的第二列包含了基于劳动力市场方法的需求侧估计值。我们发现，如果公司必须重新编写他们使用的所有 OSS 包（例如，OSS 本身不再存在，每个使用 OSS 包的公司都必须重新编写它），那么仅使用来自低收入或高收入国家的劳动力，总成本将分别在 2.59 万亿到 13.18 万亿美元之间。使用全球各地的程序员池，以大约 8.80 万亿美元的成本可以重新编写所有被广泛使用的 OSS。解读这个数字会稍微复杂一些，但仍然可行。根据 Statista（2023）的报告，2020 年（与我们数据同年）的全球软件收入为 5317 亿美元。然而，这代表的是流量，而非存量。根据政府估计软件在三年内完全折旧，我们可以做一个粗略的计算，将 2020 年使用的预装软件的全部存量价值视为 2018 年至 2020 年销售的总和，即约 1.54 万亿美元。此外，这仅代表预装软件的支出，不包括定制购买或内部开发的软件。这里，关于私营部门软件总投资的最佳可获得估计来自美国国民收入和产品账户数据（NIPA 2023）。2020 年，国家账户数据显示，美国私营公司在软件上花费了 4792 亿美元，其中 45%（2155 亿美元）是预装软件。如果我们假设世界其他地区也保持这一比率，那么公司在 2020 年使用的软件总支出约为 3.4 万亿美元（=1.54 万亿美元/0.45）。将这个粗略估计与基于全球平均工资的 OSS 需求侧价值估计（8.8 万亿美元）结合起来，意味着如果公司需要支付内部开发人员来编写他们目前免费使用的 OSS，他们将花费约 12.2 万亿美元（=3.4 万亿美元+8.8 万亿美元），是目前花费的 3.5 倍。

此处插入图一

图 1 显示了 OSS 价值在主要编程语言之间的异质性。面板 A 显示了供应侧价值，劳动力价值显示在纵轴上。我们发现，用 Go 创建的 OSS 包价值最高，如果这些 OSS 包不存在，需要从头创建的价值为 8.03 亿美元。Go 紧随其后的是 JavaScript 和 Java，分别为 7.58 亿美元和 6.58 亿美元。C 和 Typescript 的价值分别为 4.06 亿美元和 3.17 亿美元，而 Python 在主要语言中价值最低，约为 5500 万美元。JavaScript 不仅是自 2014 年以来 GitHub 上的顶级语言（GitHub, 2022），也是我们数据中价值最高的语言之一。相比之下，Python 随着时间的推移越来越受欢迎，从 2020 年 GitHub 上所有 OSS 包的第四位上升

到第二位，而它在我们的主要语言中排在最后一位。

面板 B 显示了主要编程语言的需求侧价值。基于使用产生的价值，Go 的价值是下一种语言 JavaScript 的四倍多。TypeScript（一种扩展了 JavaScript 的语言）的使用量大幅增长，从 2017 年排名前十的语言中的第十位跃升至 2020 年的第四位，这也在我们的数据中有所体现，其中 TypeScript 在需求方面是第三重要的语言。在这两种网络语言之后是 C 语言，而 Java 和 Python 则远远落后于它们。

此处插入图二

图 2 按照我们面向内部（Census）和面向外部（BuiltWith）的数据来源，对每种语言的 OSS 价值估算进行了划分。图 A 和图 B 主要关注的是 Census 数据来源的供需两侧估算。我们得到了与汇总数据（图 1）中已确立的类似模式，尽管 JavaScript 的影响要低得多。在图 A 中的 Census 供应侧方面，Java 的价值位居第二，而来自 Census 的 JavaScript 代码对总体价值的贡献却要低得多。同样，C、Python 和 TypeScript 的供应侧价值主要由 Census 决定。从需求侧的图 B 中，我们发现 Go 是面向内部代码最受欢迎的语言，而其他语言在相对意义上似乎都微不足道。

图 2 中的 C 和 D 两部分展示了 BuiltWith 数据集的供给和需求方面的数据。C 面板中的供给数据明确表明，来自 BuiltWith 样本的数据是由 JavaScript 代码驱动的，这是一个很好的验证手段，因为正如上文所述，我们在 BuiltWith 样本中重点使用了 JavaScript 包来代表开源软件。第二高的数值是由 TypeScript 产生的，这令人放心，因为它是一个包含 JavaScript 的超集。D 面板则在使用方面呈现出类似的情况，其中大部分价值来自 JavaScript，而 TypeScript 则排在第二位。其他语言对供给和需求方面的数据贡献微乎其微。总体而言，这些发现与各种语言的主要使用场景（网络编程与应用程序编程）大致相符，并且也表明产生价值的语言不一定与普通大众使用的语言完全相同。

此处插入图三

图 3 显示了使用 BuiltWith 在线数据，按 NAICS 两位行业代码划分的 OSS 需求侧价值。这可以理解为每个行业因 OSS 存在而获得的价值。价值最高的行业是“专业、科学和技术服务业”，约为 430 亿美元。“零售贸易”以及“行政、支持、废物管理和修复服务”分别构成了 OSS 需求侧外部价值的另外两个主要部分，约为 360 亿美元和 350 亿美元。相比之下，“矿业、采石业、石油和天然气开采业”、“公用事业”、“农业、林业、渔业和狩猎业”仅占价值的一小部分。后者是典型的非服务业，因此预计软件在其中发挥的作用较小。

此处插入图四

图 3 显示了供应侧（面板 A）和需求侧（面板 B）的洛伦兹曲线，以及一定比例的程序员贡献的代码仓库数量。洛伦兹曲线如果恰好落在 45 度线上，则表示程序员之间的价值分配非常均匀。相反，面板 A 显示洛伦兹曲线几乎是一条平坦的线，在最后一部分程序员处急剧上升。这意味着供应价值的分布极不均匀，比 80/20 标准更为集中。事实上，最后 5%的程序员，即 3,000 名程序员，创造了超过 93%的供应侧价值。类似地，面板 B 显示——在考虑使用量时——最后 5%的程序员创造了超过 96%的需求侧价值。总的来说，这表明极少数的程序员创造了绝大部分被公司严重依赖来创建自身代码的 OSS 代码。在面板 A 和面板 B 中，我们还可以看到，对于贡献价值最高的最后 10-15%的程序员，其贡

献的代码仓库数量也有所增加，这意味着由少数程序员产生的不均匀价值不仅仅是因为少数几个高价值的代码仓库，而是因为这少数贡献者参与了大量的代码仓库。

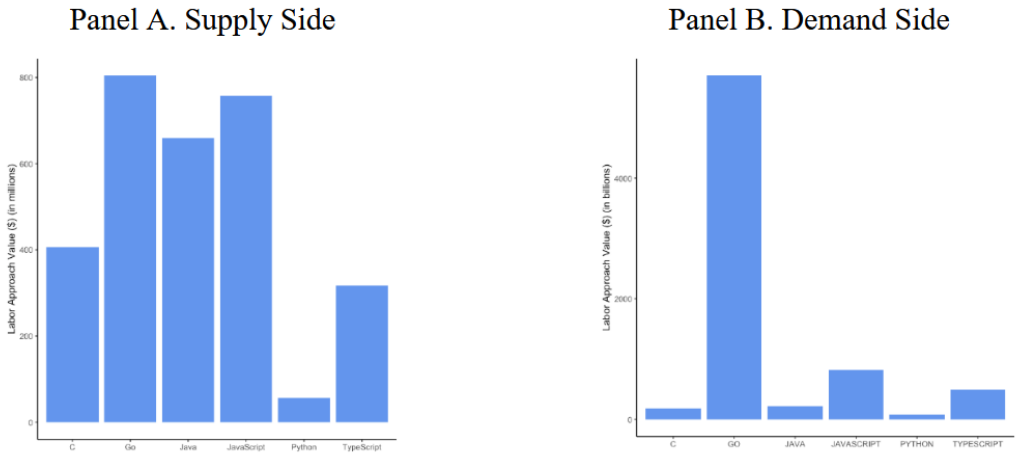
5.结论

在本研究中，我们使用两个独特的数据集：OSS 普查和 BuiltWith 数据，估计了全球广泛使用的开源软件的价值。我们不仅能够估计现有代码的供应侧价值（例如，重新编写每个广泛使用的 OSS 包一次所需的成本），还能估计私营经济的需求侧价值（例如，每个使用某个 OSS 包的公司重新编写它所需的成本）。虽然我们并未关注 OSS 的长尾部分，但我们认为这是我们研究的另一个贡献，因为专注于广泛使用的 OSS 可以让我们更精确地理解 OSS 创造的价值，而不仅仅是衡量所有 OSS 的重置成本（这会高估真实价值，因为许多 OSS 项目并未在生产代码中使用）。然而，尽管我们基于广泛的使用数据强调了 OSS 在我们社会中具有的巨大价值，但要识别全球 100%的 OSS 使用情况是不可行的，因此，我们的需求侧估计很可能低估了真实价值。

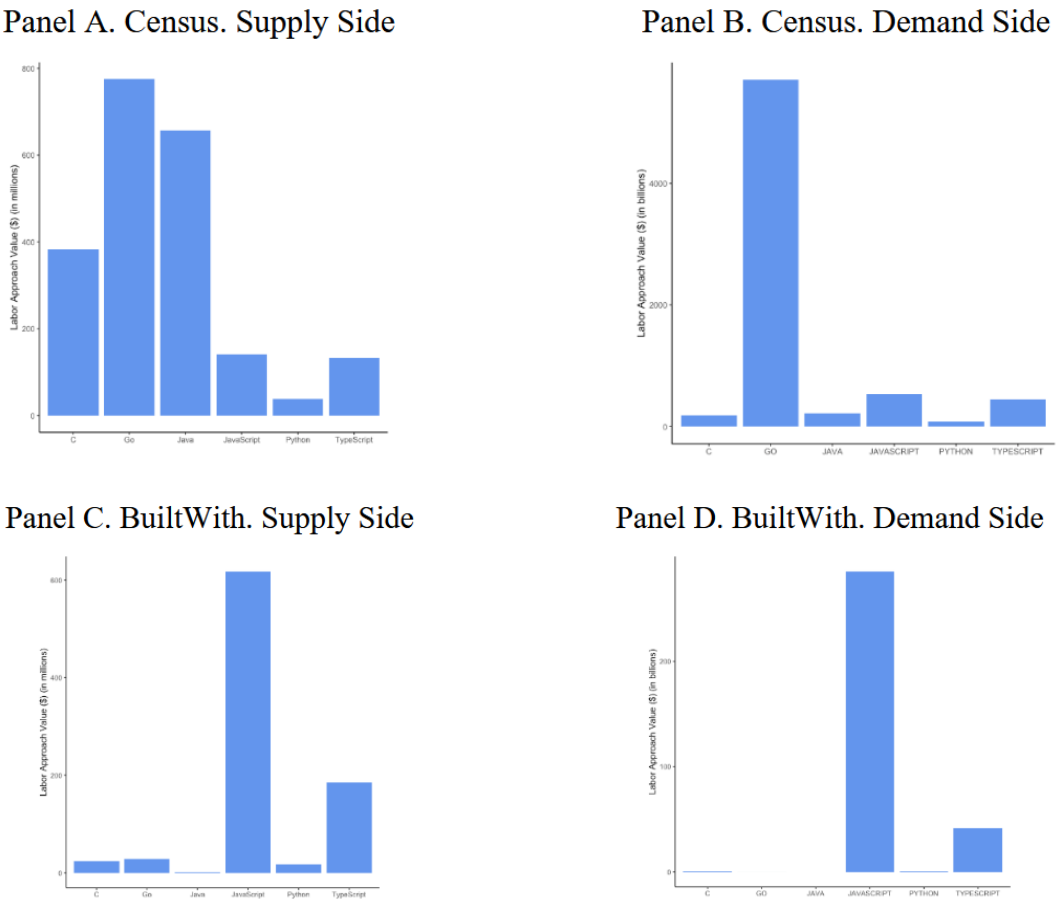
根据使用量进行调整后，我们发现当使用来自世界各地的程序员时，OSS 的需求侧价值高达 8.8 万亿美元，根据我们仅雇佣来自低收入或高收入国家的程序员的不同情况，该数值会有一些差异。不同编程语言之间的价值存在显著的异质性，代码是面向内部（即用于创建待售产品）还是面向外部（即用于公司网站）也存在差异。排名前 6 的编程语言创造了 84%的需求侧价值。我们还展示了不同行业之间的显著异质性，以及程序员自身价值贡献的异质性。超过 95%的需求侧价值仅由 5%的程序员创造，并且这些程序员不仅为少数几个广泛使用的项目做出贡献，而且比处于价值分布低端的程序员参与了更多的项目。

总之，我们的结果表明，尽管由于价格为零且仅使用公共数据难以衡量数量，导致 OSS 的价值通常通过直接测量显示为零，但它对经济做出了巨大贡献。我们的研究不仅为未来对 OSS 的研究，也为所有信息技术及其对全球经济日益增长的影响的研究奠定了基础。

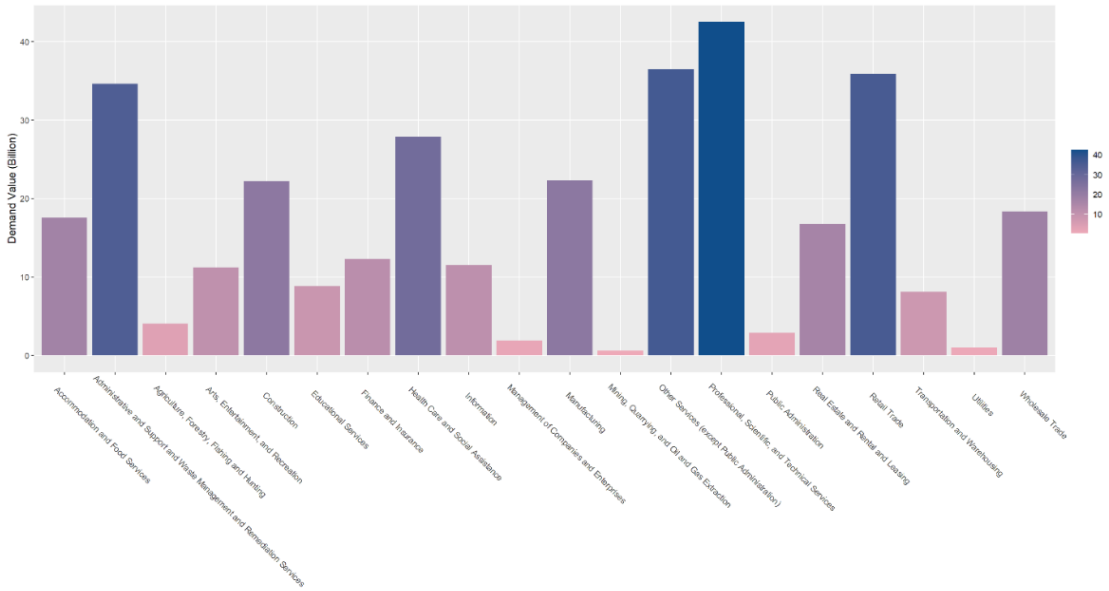
参考文献　略 不做翻译



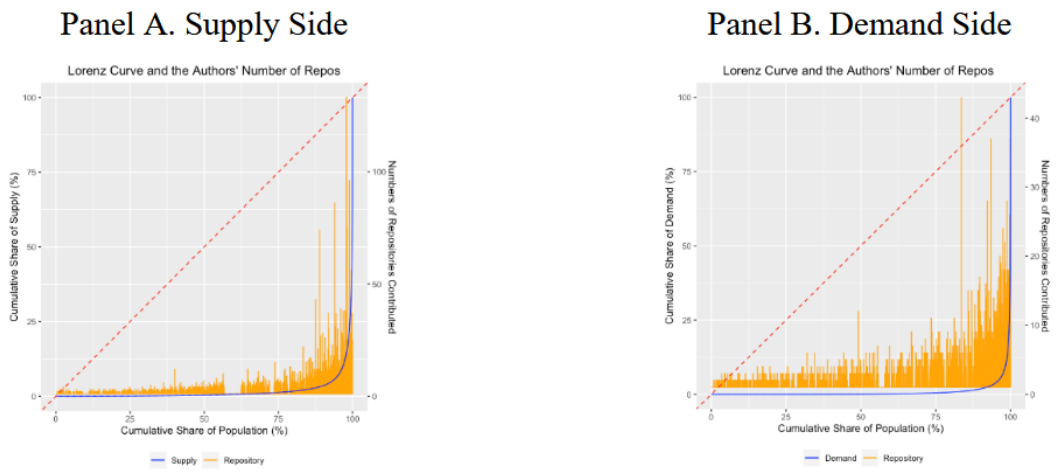
图一 开源软件的价值：顶级语言



图二 开源在排名前五的语言（包括 Go 语言）以及数据源中的价值



图三 开源技术在各行业的价值，从网站来看



图四 开源价值创造的分布

Table 1 Descriptive statistics on lines of code and usage

	Sum	Mean	SD	Obs
Panel A: Census				
Lines of Code – All packages	261,653,728	142,203.1	887,937.2	1,840
Lines of Code – Top 5 Languages and Go	189,673,184	113,712.9	702,832.1	1,668
Usage – All packages	2,709,155	1472.4	2,167.9	1,840
Usage – Top 5 Languages and Go	2,497,785	1497.5	2,228.8	1,668
Panel B: BuiltWith				
Lines of Code – All packages	82,504,613	111,342.3	613,488.1	741
Lines of Code – Top 5 Languages and Go	58,664,935	79,925.0	354,415.0	734
Usage – All packages	142,794.4	192.7	733.4	741
Usage – Top 5 Languages and Go	142,751.2	194.5	736.6	734

Table 2 The labor-market value of open source

	Labor Supply	Labor Demand
Wage: Low	$1.22 Billion	$2.59 Trillion
Wage: Global	$4.15 Billion	$8.80 Trillion
Wage: High	$6.22 Billion	$13.18 Trillion

Note. The high wage scenario is based on the US average wage and the low wage scenario is the Indian average wage for programmers in 2020. The global wage is an average wage from the countries in Table A4, weighted according to their contributions to OSS. These estimates include only languages from software classified in bucket 1 (see Table A1).

表一　每个桶内的语言

Panel A: Bucket 1 - Languages	
Mark-Up Language	BIBTEX
Mark-Up Language	COLDFUSION HTML
Mark-Up Language	DOCBOOK XML
Mark-Up Language	HAML
Mark-Up Language	HTML
Mark-Up Language	HXML
Mark-Up Language	JAVAEE XML
Mark-Up Language	MARKDOWN
Mark-Up Language	MASON
Mark-Up Language	MXML
Mark-Up Language	RELAX-NG COMPACT
Mark-Up Language	RHTML
Mark-Up Language	TEX
Mark-Up Language	XML
Mark-Up Language	XQUERY
Mark-Up Language	YAML
Programming Language	ABNF
Programming Language	ACTIONSCRIPT
Programming Language	ADA
Programming Language	APPLESCRIPT
Programming Language	ARDUINO
Programming Language	ASPECTJ
Programming Language	ASPX-CS
Programming Language	ASPX-VB
Programming Language	AWK
Programming Language	C
Programming Language	C#
Programming Language	CHARMCI
Programming Language	CLOJURE
Programming Language	COFFEESCRIPT
Programming Language	COMMON LISP
Programming Language	CSS
Programming Language	CUDA
Programming Language	CYTHON
Programming Language	D
Programming Language	DART
Programming Language	DELPHI
Programming Language	EASYTRIEVE
Programming Language	EC
Programming Language	ELIXIR
Programming Language	ELM
Programming Language	EMACSLISP
Programming Language	ERLANG
Programming Language	F#
Programming Language	FISH
Programming Language	FORTH
Programming Language	FORTRAN
Programming Language	FORTRANFIXED

Programming Language	GAP
Programming Language	GHERKIN
Programming Language	GLSL
Programming Language	GO
Programming Language	GRAPHVIZ
Programming Language	GROOVY
Programming Language	HASKELL
Programming Language	HAXE
Programming Language	IDL
Programming Language	JAVA
Programming Language	JAVA SERVER PAGE
Programming Language	JAVASCRIPT
Programming Language	KOTLIN
Programming Language	LESSCSS
Programming Language	LIQUID
Programming Language	LIVESCRIPT
Programming Language	LLVM
Programming Language	LOGOS
Programming Language	LUA
Programming Language	MATHEMATICA
Programming Language	MINISCRIPT
Programming Language	MODULA-2
Programming Language	NASM
Programming Language	NIX
Programming Language	OBJECTIVE-C
Programming Language	OBJECTIVE-J
Programming Language	OCAML
Programming Language	OPENEDGE ABL
Programming Language	PAWN
Programming Language	PERL
Programming Language	PHP
Programming Language	PL/PGSQL
Programming Language	POSTSCRIPT
Programming Language	POVRAY
Programming Language	PROLOG
Programming Language	PROPERTIES
Programming Language	PUPPET
Programming Language	PYTHON
Programming Language	REASONML
Programming Language	REBOL
Programming Language	REDCODE
Programming Language	REXX
Programming Language	RUBY
Programming Language	RUST
Programming Language	S
Programming Language	SASS
Programming Language	SCALA
Programming Language	SCILAB
Programming Language	SCSS
Programming Language	SLIM
Programming Language	SMALLTALK
Programming Language	SOLIDITY
Programming Language	STANDARD ML
Programming Language	SWIFT
Programming Language	SWIG
Programming Language	TADS 3

Programming Language	TCL
Programming Language	THRIFT
Programming Language	TRANSACT-SQL
Programming Language	TREETOP
Programming Language	TYPESCRIPT
Programming Language	VB.NET
Programming Language	VBSCRIPT
Programming Language	VCL
Programming Language	VIML
Programming Language	WEB IDL

Panel B: Bucket 2 – Auxiliary Languages

Assembler/Compiler/Interpreter/Macro Processors	GAS
Assembler/Compiler/Interpreter/Macro Processors	M4
Assembler/Compiler/Interpreter/Macro Processors	RAGEL IN RUBY HOST
Configuration	CMAKE
Configuration	INI
Configuration	MAKEFILE
Configuration	NGINX CONFIGURATION FILE
Configuration	NSIS
Configuration	SQUIDCONF
Configuration	TERRAFORM
Configuration	TOML
Formatting	GROFF
IDE	NETBEANS PROJECT
Template Engine	CHEETAH
Template Engine	GENSHI
Template Engine	PUG
Template Engine	SMARTY
Template Engine	VELOCITY
Terminal/Batch	ANT
Terminal/Batch	APACHECONF
Terminal/Batch	BASH
Terminal/Batch	BATCHFILE
Terminal/Batch	MAVEN
Terminal/Batch	POWERSHELL
Terminal/Batch	RPMSPEC
Terminal/Batch	SINGULARITY
Terminal/Batch	TCSH
Translational	GETTEXT CATALOG

Panel C: Bucket 3 - Data

Data	BNF
Data	DIFF
Data	DTD
Data	E-MAIL
Data	JSON
Data	PROTOCOL BUFFER
Data	RESTRUCTUREDTEXT
Data	TEXT ONLY
Data	XSLT

表 A2　全球工资最高的 30 个国家

Table A2 Top 30 countries included for the global wage

Country
United States
China
Germany
India
United Kingdom
Brazil
Russia
France
Canada
Japan
South Korea
Netherlands
Spain
Poland
Australia
Sweden
Ukraine
Italy
Switzerland
Indonesia
Taiwan
Colombia
Argentina
Mexico
Norway
Belgium
Denmark
Finland
Vietnam
Austria

表 A3 使用桶 1 和桶 2 中语言的开源劳动力市场价值

Table A3 The labor-market value of open source using languages in buckets 1 and 2

	Labor Supply	Labor Demand
Wage: Low	$1.23 Billion	$2.60 Trillion
Wage: Global	$4.18 Billion	$8.84 Trillion
Wage: High	$6.26 Billion	$13.24 Trillion

Note. The high wage scenario is based on the US average wage and the low wage scenario is the Indian average wage for programmers in 2020. The global wage is an average wage from the countries in Table A4. The estimates include only languages from buckets 1 and 2 (see Table A1).

表 A4　使用桶 1、桶 2 和桶 3 中语言的开源劳动力市场价值

Table A4 The labor-market value of open source using languages in buckets 1, 2, and 3

	Labor Supply	Labor Demand
Wage: Low	$1.88 Billion	$3.52 Trillion
Wage: Global	$6.41 Billion	$11.96 Trillion
Wage: High	$9.59 Billion	$17.91 Trillion

Note. The high wage scenario is based on the US average wage and the low wage scenario is the Indian average wage for programmers in 2020 The global wage is an average wage from the countries in Table A4. The estimates include only languages from buckets 1, 2, and 3 (see Table A1).

表 A5　商品篮子——等效的开源和专有的软件

Table A5 Goods Basket – Equivalent Open Source and Proprietary Software

Open Source Software	Proprietary Software
Apache Http Server	Windows Server 2008
Audacity	Adobe Audition
Blender	Autodesk Maya
Elasticsearch	Amazon Kendra
FileZilla	SmartFTP
FreeCAD	AutoCAD
GIMP	Adobe Photoshop
GNU Octave	MATLAB
GnuCash	QuickBooks
KeePass	1Password
LibreOffice	Microsoft Office Suite
MariaDB server	Microsoft SQL Server
Metabase	Tableau
MySQL	Oracle MySQL
OpenVPN	ExpressVPN
PSPP	SPSS
Redis	Redis Enterprise
TensorFlow	TensorFlow Enterprise
VirtualBox	VMware Workstation
VLC Media Player	CyberLink PowerDVD

表 A6　使用桶 1、桶 2 和桶 3 中语言的开源商品市场价值

Table A6 The goods-market value of open source using languages in buckets 1, 2, and 3

	Goods-Demand Bucket 1	Goods-Demand Bucket 1-2	Goods Demand, Bucket 1-3
Wage: Low	$177 Million	$179 Million	$242 Million
Wage: Global	$177 Million	$179 Million	$242 Million
Wage: High	$177 Million	$179 Million	$242 Million

Note. The high wage scenario is based on the US average wage and the low wage scenario is the Indian average wage for programmers in 2020 The global wage is an average wage from the countries in Table A4. The estimates include only languages from buckets 1, 2, and 3 (see Table A1).

附录 A　商品市场估值方法

作为另一种估算方法，我们不使用劳动力重置成本，而是使用商品重置价值方法。我们识别出几个有类似闭源、付费替代品的 OSS 包，并考虑如果所有免费 OSS 的商业用户都必须用付费替代品替换该软件将会产生的成本（类似于 Greenstein 和 Nagle（2014）以及 Murciano-Goroff 等人（2021）对网页服务器所做的计算）。然后，我们可以使用这个替代的 pp 值，结合上面的 qq 值，来估算 OSS 的商品重置价值。该方法建立在 Nordhaus（2006 年，第 146 页）的建议之上，他说："……市场和非市场商品及服务的价格应根据可比较的市场商品和服务来估算。"我们不期望这两种方法的估算结果相似。恰恰相反，这两种方法的价值差异很大，因为后一种商品市场方法假设以固定价格多次销售商品，并且该固定价格通常低于从劳动力方面重新创建所有包所估算的总价值。这些估算值之间的差异本质上是以下两种情形结果之差：一家公司介入，重新创建缺失的 OSS 包，然后以付费价格出售它们；而所有公司都需要从头开始重新创建这些包。

通过商品市场方法，我们设想的情景仍然是：我们生活在一个没有 OSS 的世界，但它必须由一家公司重新创建，然后该公司对当前免费的商品收取费

用。为了通过商品市场方法评估 OSS 的价值，我们创建了一个篮子，里面装有等价的专有替代品，这些替代品在公开市场上作为 OSS 产品的替代品定价。该方法与先前文献中使用的方法一致（Greenstein 和 Nagle，2014；Murciano-Goroff 等人，2021），尽管这两项研究都只关注单一商品（网页服务器），而不是一个篮子。由于没有现成的 OSS 专有等价物数据库，我们基于对 OSS 流行程度的主观感知进行搜索，然后为它们寻找付费的、闭源的替代品。由此产生的包含 20 个具有专有替代品的 OSS 包的篮子，很好地代表了 OSS 的多样性。该软件范围涵盖从媒体和设计软件到统计分析程序，再到数据库管理和网页服务器软件。

根据我们的 OSS 等价替代专有商品篮子，我们获得了每个专有软件等价物的价格，并计算了篮子中每个 OSS 产品的 COCOMO 劳动力市场供应侧价值，我们将其用作专有软件 COCOMO 劳动力供应侧价值的代理（例如，支付程序员从头开始编写该专有软件的成本），因为缺少专有软件的代码。然后，我们计算篮子的平均 COCOMO 劳动力供应侧价值 $V_{S,Basket}$ 和篮子的平均价格 $P_{S,Basket}$。由于我们知道所有 OSS 的劳动力市场供应侧价值 V_S^{Labor}，我们可以建立以下方程，并通过简单的缩放变换得到所有 OSS 的商品市场价格 P_S：

$$\frac{P_S}{V_S^{Labor}} = \frac{P_{S,Basket}}{V_{S,Basket}}$$

$$P_S = \frac{V_S^{Labor}}{V_{S,Basket}} P_{S,Basket} \tag{4}$$

这得出以下的商品市场供应侧价值：

$$V_S^{Goods} = P_S * 1. \tag{5}$$

然后，我们可以得到相应的需求侧商品市场价值如下：

$$V_D^{Goods} = P_S * Q. \tag{6}$$

从商品市场的角度来看，我们考虑如果一家公司生产所有现有的广泛使用的 OSS，然后将其作为产品销售给客户，该公司将收取的等价价格。

表 A6 第 3 列显示，需求侧的价值在不同桶之间（与雇佣程序员的原籍国无关）介于 1.77 亿美元到 2.44 亿美元之间。自然地，商品市场价值将远小于劳动力需求价值，因为这家虚构的公司将通过一次生产软件然后卖给许多客户，以相对较低的价格获利。此外，虽然 Greenstein 和 Nagle（2014）以及 Murciano-Goroff 等人（2021）仅关注一种特定类型的软件（网页服务器），但我们试图将他们的方法扩展到多种商品。然而，在这种情况下，数据的局限性成为了更为严重的制约因素，这使得通过这种方式来估算价值变得异常困难，同时还需要做出大量的假设。比起采用单纯的货物市场方法，这一需要借助劳动力市场的方法需要确定估算的规模，这最终会导致对开源软件价值的严重低估。此外，专有软件的定价策略会受到市场需求的影响。当货物市场方法扩展到多种货物时，一个强烈的隐含假设是，我们所涵盖的软件产品和我们的样本开源软件的市场需求是相似的，因此它们会导致相应的价格。

另一种更简单的、不涉及代码行数且不依赖劳动力市场方法进行缩放的商品市场粗略估算法，只需将参考商品的价格乘以使用量即可。我们可以选取专有商品篮子中的最低价、平均价和最高价，如参考价格向量

p=(69.99,1610.17,5800)所示，然后将其与 Census 和 BuiltWith 联合样本的使用量（表 1）相乘。基于这些估算的专有价格假设，商品市场需求侧价值的范围将在 0.2 万亿至 16.5 万亿美元之间（最低价至最高价），平均价格得出的价值估计值为 4.5 万亿美元。然而，尽管这种估算方法比劳动力市场方法产生的方差稍大，但它存在固有的缺陷，即简单地假设每个开源产品的估算价格都基于参考价格而相同。我们进一步认为，这种非常粗略的估算是橘子和苹果的比较，因为我们篮子中的商品是功能齐全、独立的软件包，而 Census 和 BuiltWith 数据集中的包由应用程序库组成，通常比这种独立的包要小。

鉴于由于缺乏商品市场方法的数据而必须做出的所有复杂性和假设，我们将重点放在正文中强调的劳动力成本方法上，但为了完整起见，我们在此处包含此方法。

主要观点：

这篇论文以实证的形式研究了“广泛使用的开源软件在需求侧的系统估值”这一问题，其所依托的现实背景是：开源软件如今已经十分流行，但其经济价值又难以估计，因为传统经济学的“价格×数量”方法不适用于开源的价值估计。

作者在 Census 和 BuiltWith 这两个互补数据集中收集数据，以 COCOMO II 模型、洛伦兹曲线以及各类实证模型为工具，在一番探索后得出了结果：无论是供给侧，还是需求侧，无论是在发达国家，还是在发展中国家，开源软件都能为使用者带来巨大的经济收益，只不过其价值是高度集中在少数语言和少数开发者处的。此外，开源软件经济价值的分布具有行业异质性，并非每个行业都能从使用开源软件中获得最大化的收益（专业、科学与技术服务行业的价值最高）。

这份研究弥补了前人的文献中“无法兼顾深度与广度”的缺陷，也为企业和政策制定者执行的决策提供了理论依据。